

Benchmarks are Not Enough: RAMP for Reexamining Agentic Models in Production Systems

Yipeng Ouyang¹, Xin Huang¹, Bingjie Liu¹, Zhongchun Zheng¹, Yuhao Gu¹, Xianwei Zhang¹

¹ Sun Yat-sen University, Guangzhou, China

{ouyyp5, huangx385, liubj8, zhengzhch3, guyh9}@mail2.sysu.edu.cn, {zhangxw79}@mail.sysu.edu.cn

2RampGraywhite

Dimension		Key Finding
	Model Capability	Even the strongest model achieves only around 90% task success rate, while the average success rate across models remains below 70% .
	Cost Efficiency	The cost-efficiency gap between models can exceed two orders of magnitude (up to 100×) under realistic production workloads.
	Benchmark Reliability	Up to 72% of tasks that succeed on conventional benchmarks fail under production-scale workflow execution.
	Runtime Robustness	Production execution introduces 3–5× higher failure rates than static benchmarks.
	Failure Attribution	More than 60% of execution failures originate from system interaction rather than core reasoning.
	Long-Horizon Degradation	Long-horizon workflows reduce task completion rates by over 40% .
	Workflow Sensitivity	Workflow-level assessment reveals up to 2.8× performance divergence between models with similar benchmark scores.
	Runtime Recovery	Runtime recovery capability improves production success rates by up to 35% .
	Environment Dependency	Environment dependency issues account for over 45% of software engineering task failures.

Abstract

Large language model (LLM) agents are rapidly evolving from coding assistants into autonomous software engineering systems. However, existing evaluation methodologies remain largely centered on static, isolated, and short-horizon benchmarks that fail to capture the dynamic complexity of real-world production workflows. As a result, benchmark performance often poorly reflects practical capability under realistic runtime environments involving long execution chains, tool interactions, dependency management, and iterative feedback loops. We thus present RAMP (Runtime Assessment of Models in Production), a production-grounded assessing framework built upon the *YatCC* integrated platform, which has been extensively used in practice. RAMP provides a unified assessment infrastructure supporting heterogeneous LLM providers and agent SDKs through standardized API interfaces and runtime orchestration. RAMP introduces realistic compiler construction workloads characterized by long-horizon execution, strict serial dependencies, iterative feedback cycles, and complex toolchain interactions. In addition, RAMP incorporates a *Resurrection* mechanism that decomposes cascading workflow failures into recoverable diagnostic stages, enabling fine-grained analysis of execution behaviors and intermediate recovery capability. The design further introduces utility-oriented multi-dimensional metrics that jointly measure both outcome quality and process efficiency. Across 22 model **configurations** and **four** agent backends, we demonstrate

that **serial** assessment reveals failure patterns invisible to static benchmarks, that *resurrection* recovers substantial downstream diagnostic signal (median 73.65-point pipeline score recovery), and that process cost is decoupled from outcome ($R^2 = 0.007$), providing diagnostic rather than predictive value. RAMP thus establishes a new paradigm for agent **evaluation** that moves beyond synthetic benchmarks toward continuous, runtime-observable, production-grounded assessment.

Keywords

agent model **assessment**, production systems, compiler construction, serial dependency, resurrection mechanism

1 Introduction

Large language models (LLMs) have rapidly evolved from conversational assistants into autonomous software engineering agents capable of planning, implementing, debugging, and deploying increasingly complex software stacks and infrastructure workflows [? ? ?]. Recent agentic models have demonstrated strong capabilities in repository-level code generation, multi-file reasoning, iterative tool invocation, and workflow orchestration. However, despite these advances, the evaluation methodologies for such systems remain largely rooted in a *benchmark-driven paradigm* consisting of isolated, static, and short-horizon tasks, where each evaluation

instance is executed independently and the agent state is reset after every attempt.

This evaluation paradigm exhibits a fundamental mismatch with real-world production engineering environments. In practice, production software and infrastructure engineering involve long-horizon iterative workflows spanning multi-file and cross-module reasoning, dependency management, runtime debugging, toolchain coordination, deployment validation, continuous integration pipelines. Real production systems must further operate under imperfect runtime conditions including environment instability, resource constraints, asynchronous execution, and cascading failure propagation. Existing static benchmarks fail to capture these dynamics—they primarily measure instantaneous coding task completion rather than sustained engineering capability under evolving runtime conditions. Moreover, conventional benchmarks provide limited visibility into operational efficiency, execution stability, recovery behavior, and cost-performance tradeoffs, all of which are critical factors in practical deployment environments.

As a result, strong benchmark performance does not necessarily imply practical engineering utility. Models that perform well on benchmark-style coding may still **fail** in realistic production environments where runtime correctness, execution robustness, and adaptive recovery become critical. In many cases, failures originate not from insufficient code generation ability, but from unstable tool interaction, environment inconsistency, cascading execution errors, and inefficient runtime orchestration. Existing benchmark paradigms largely overlook these system-level behaviors because they evaluate isolated task completion rather than continuous execution under evolving runtime conditions. We therefore argue that the field must move *beyond static benchmarks* toward production-grounded *assessment frameworks*—systems that gauge agentic models in realistic, multi-stage engineering environments with persistent runtime state, controlled intermediate state injection, multi-dimensional metrics, and systematic failure diagnosis. Such assessment should not only measure whether a task succeeds, but also characterize how the system behaves throughout the execution process.

To this end, we present RAMP (Runtime Assessment of Models in Production), a production-grounded assessment framework built upon the *YatCC* integrated serving platform. RAMP provides a unified runtime assessment infrastructure supporting heterogeneous LLM providers and agent SDKs, including OpenHands, Claude Code, Codex CLI, Kimi CLI, through standardized API access and orchestration services via **AIHUB**. Unlike conventional benchmark pipelines, RAMP assesses models using realistic long-horizon compiler construction workloads organized as serial dependency chains, reflecting practical software engineering and infrastructure development processes. *The framework further introduce Progressive, a representative workload enabling systematic analysis of scaffolding effects and execution complexity under production-style workflows.* A key design of RAMP is the Resurrection mechanism, which decomposes cascading workflow failures into recoverable diagnostic stages through controlled intermediate-state restoration. This mechanism substantially improves runtime observability and enables fine-grained analysis of downstream execution behavior that would otherwise be hidden by early-stage failures. In addition,

RAMP introduces utility-oriented multi-dimensional metrics that jointly **evaluate** outcome quality, execution efficiency, runtime cost, workflow robustness, recovery capability, and deployment-oriented operational behavior.

In summary, the paper makes the following contributions:

- (1) We propose RAMP as a production-oriented assessment *framework*, not merely another benchmark, that moves beyond synthetic isolated tasks toward persistent, runtime-observable evaluation of agentic models under realistic engineering environments.
- (2) We construct realistic compiler construction workloads with strict serial dependencies, verifiable intermediate states, and two difficulty levels that systematically quantify the impact of scaffolding and execution complexity in agentic software engineering tasks.
- (3) We introduce a unified assessment infrastructure supporting heterogeneous LLMs and agent frameworks while introducing utility-oriented metrics that characterize not only task outcomes, but also runtime efficiency, failure patterns, and deployment-relevant operational cost.
- (4) We provide empirical evidence across 22 model configurations **and four** agent backends, demonstrating that serial assessment reveals failure patterns invisible to static benchmarks and that resurrection-based recovery substantially improves downstream diagnostic observability.

2 Background and Motivation

2.1 YatCC and Compilation Tasks

(Introduce YatCC and Compiler)

TODO:

YatCC is a one-stop serving platform that supports application development, deployment, execution, and management across heterogeneous computing environments through unified runtime abstractions.

Compiler construction tasks further represent a highly representative production-scale workload because they require deep integration across code generation, build systems, debugging workflows, dependency management, and iterative execution feedback. (LLVM)

Intermediate artifacts and verification. Each task produces a well-defined intermediate artifact that serves as both the **evaluation** object for the current task and the input dependency for the next: Task 1 produces a *token stream*; Task 2 produces an AST/ASG; Task 3 produces *LLVM IR*; Task 4 produces *optimized IR*; Task 5 produces *RV64 assembly*. Scoring is performed by dedicated test suites that execute the agent’s code against comprehensive test cases. Each task’s score ranges from 0 to 100, **with a passing threshold of 60%.**

Importantly, these workloads have been validated through years of practical use in compiler construction education at Sun Yat-sen University, ensuring their authenticity as real engineering tasks rather than synthetic benchmarks.

2.2 Production-level Serving

(State why YatCC and Compiler are best as assessment)

Applications deployed on YatCC interact with workflow runtimes, containerized execution environments, external toolchains,

Table 1: Benchmark comparison. ✓ = supported, — = not supported. *Serial* = tasks form a dependency chain; *Exec.* = execution-grounded scoring; *Resurr.* = resurrection mechanism; *Process* = process metrics as first-class output; *Artifact* = intermediate artifact continuity tracked.

Benchmark	Serial	Exec.	Resurr.	Process	Artifact	Tasks
SWE-bench [?]	—	✓	—	—	—	2.3K
AgentBench [?]	—	✓	—	—	—	8 envs
GAIA [?]	—	—	—	—	—	466
OSWorld [?]	—	✓	—	—	—	369
WebArena [?]	—	✓	—	—	—	812
Terminal-Bench [?]	—	✓	—	—	—	50
SkillsBench [?]	—	✓	paired	—	—	1.5K
RE-Bench [?]	—	✓	—	partial	—	7
RAMP	✓	✓	✓	✓	✓	6

and resource-managed service layers within a realistic operational setting. Such workloads naturally introduce challenges that are often absent from conventional benchmarks, including long-horizon execution, multi-stage tool invocation, asynchronous task coordination, failure recovery, environment dependency management, and runtime observability. As a result, YatCC enables comprehensive assessment not only of functional correctness, but also of runtime robustness, orchestration efficiency, operational stability, and adaptability under practical deployment conditions. These characteristics make YatCC particularly suitable for evaluating production-oriented systems beyond static benchmark environments.

Unlike conventional programming benchmarks that primarily focus on localized code synthesis, compiler-related tasks involve large codebases, multi-stage execution pipelines, strict correctness requirements, and complex coordination across parsing, optimization, code generation, linking, and runtime validation. Such workloads require continuous interaction with external compilation and testing tools, iterative diagnosis of execution failures, and consistent management of evolving project states over long execution horizons. These properties closely resemble real-world software engineering and systems development environments, making compiler construction workloads an effective proxy for evaluating the practical capability, reliability, and runtime behavior of modern intelligent systems in production settings.

2.3 Motivation and Challenges

(How to enable assessment in production)

Existing agent benchmarks suffer from three fundamental limitations: (1) *task isolation*—each task is independent, preventing *assessment* of cross-task dependency effects; (2) *outcome-centric evaluation*—only final scores are reported, with no visibility into process cost, failure patterns, or recovery behavior; (3) *infrastructure coupling*—each benchmark requires custom evaluation infrastructure, making cross-benchmark comparison and model-backend matrix evaluation impractical. (4) *unlimited computational resources, time, and tool call budgets*

Table 1 positions RAMP against existing benchmarks across five dimensions. RAMP is the only one that combines serial dependency, execution-grounded scoring, resurrection, process metrics, and artifact continuity tracking. While SWE-bench and SkillsBench

share execution-grounding and paired evaluation (respectively), neither addresses serial dependency or intermediate artifact flow. RE-Bench records partial process metrics but lacks resurrection and artifact tracking. The combination of these five dimensions enables RAMP to measure *evaluation* axes that no existing benchmark captures simultaneously.

To effectively assess the practical capability of agentic models in production-grade systems, the following key capabilities are necessitated:

A unified runtime infrastructure for heterogeneous model evaluation. Production environments typically involve diverse models, APIs, execution frameworks, workflow engines, and deployment backends. A practical assessment platform should therefore provide a unified infrastructure that enables reproducible and scalable evaluation across heterogeneous models and runtime systems. Such an infrastructure should support standardized service interfaces, workflow orchestration, environment provisioning, dependency isolation, and runtime management, allowing different models and execution frameworks to be evaluated under consistent operational conditions. The infrastructure should also enable closed-loop assessment pipelines for automated testing, execution tracing, error diagnosis, result aggregation, and replayable experimentation, which helps better capture the evolving behavior of agentic systems under complex and changing production workloads.

Realistic workloads and multi-dimensional assessment modes. Conventional benchmarks often focus on isolated tasks with short execution horizons, which fail to capture the characteristics of real production systems. Production-grade assessment instead requires realistic workloads involving long-running workflows, external tool invocation, iterative execution feedback, environment interaction, failure recovery, and multi-stage task coordination. Moreover, assessment should support multiple evaluation modes, including functional correctness, execution efficiency, robustness under runtime perturbations, adaptability to dynamic environments, and operational scalability under concurrent workloads.

End-to-end observability and measurable runtime metrics. Evaluating agentic systems in production settings requires comprehensive runtime visibility beyond task success rates alone. The assessment framework should provide observable metrics spanning workflow execution, tool interaction, runtime behavior, resource

consumption, latency characteristics, failure propagation, and execution stability. Such observability is essential for understanding not only whether a task succeeds, but also how the system behaves under realistic operational conditions, enabling deeper analysis of reliability, orchestration capability, and production readiness.

Together, these requirements highlight that **assessing agentic models in production systems is fundamentally a systems-level problem rather than merely a benchmark-level evaluation task**. Effective assessment therefore necessitates integrated runtime infrastructure, realistic workload modeling, comprehensive observability, and scalable orchestration mechanisms.

3 Design and Implementation

RAMP provides a unified assessment architecture that decouples workload definition from execution infrastructure, supports heterogeneous model providers and agent frameworks through a standardized API layer, and records both outcome and process metrics as first-class analytical objects.

3.1 Assessment Framework

RAMP consists of five major components (Figure 1):

Task Orchestrator. Responsible for workload scheduling, dependency resolution, resurrection triggering, and evaluation pipeline management. The orchestrator maintains the serial task chain state, monitors per-task pass/fail status, and triggers resurrection when a task falls below the 60% passing threshold.

Agent Runtime Layer. Supports multiple agent frameworks through a unified interface, each representing a distinct interaction paradigm:

OpenHands SDK: Full agent framework with unlimited self-loop iteration, skill injection, and MCP tool integration. Claude Code: File I/O and command execution agent with multi-turn conversation. Codex CLI: Autonomous coding agent with sandboxed execution. Kimi CLI: Conversational coding agent with plan management and background task execution.

All backends share a common task context format and produce standardized result structures, enabling direct cross-backend comparison.

Model Access Layer (AIHUB). A unified API gateway (<https://aihub.arcsysu.cn>) providing standardized OpenAI-compatible access to 21 large language models spanning proprietary and open-weight families. This design eliminates API-specific integration overhead and ensures that evaluation differences reflect model and scaffold capabilities rather than infrastructure variability.

Execution Environment. Each evaluation run operates in an isolated workspace containing a complete copy of the target repository with all dependencies (ANTLR, LLVM 14, pybind11) pre-installed. For *Progressive*, each run additionally launches a fresh container environment that is destroyed after completion, ensuring zero cross-run contamination. The workspace is configured via CMake with a fixed toolchain; scoring scripts are deterministic.

Observation and Leaderboard Layer. Collects and aggregates *evaluation* results, computes utility-oriented metrics, and maintains dynamic leaderboards. All raw evaluation data (scores, logs, process metrics) is preserved for reproducibility.

figures/fig1_sys_arch.png

Figure 1: The RAMP assessment framework architecture. Five layers from top to bottom: Observation & Leaderboard, Task Orchestrator, Agent Runtime, Model Access (AIhub), and Execution Environment. *YATCCYATCC-Hard*

3.2 Assessment Tasks

RAMP grounds its *evaluation* in compiler construction, a domain chosen for its strict causal dependencies, well-defined intermediate artifacts, and objective automated scoring. Compiler engineering provides an ideal *evaluation* substrate because: (1) the pipeline has *strict causal dependencies*—each stage consumes the output of its predecessor; (2) intermediate artifacts are *well-defined and verifiable*—token streams, parse trees, IR modules, and assembly files have clear structural specifications; (3) the project includes *automated scoring scripts* with comprehensive test suites.

3.2.1 Workloads: Compiler Construction as Realistic *Evaluation*. We provide a *representative workload*:

Progressive: Scaffolded Implementation. The original *Incremental workload* [?] organizes six tasks along the compiler construction pipeline (Table 2). Each task provides *scaffolded starter code*—the agent fills in missing implementation details within a provided framework. Crucially, *Progressive* persists the full execution context and runtime environment across all tasks, simulating the real-world agentic workflow where agents operate within a single, unified environment to complete tasks sequentially. This design eliminates artificial context resets between tasks, providing a more realistic evaluation of continuous system development capabilities. This workload measures an agent’s ability to *complete a partially-implemented system incrementally*.

3.2.2 Resurrection Protocol. The *Resurrection* mechanism addresses the fundamental evaluation challenge of serial workloads: when an agent fails at Task k , downstream tasks become unreachable, and

Table 2: RAMP task chain. Each task consumes the output artifact of its predecessor, forming a strict serial dependency.

Task	Name	Objective	Input Artifact
0	Environment Setup	Build & verify toolchain	—
1	Lexer	Tokenize C source code	Preprocessed source
2	Parser	Construct AST/ASG	Token stream (from T1)
3	IR Generation	Emit LLVM IR	AST/JSON (from T2)
4	IR Optimization	Implement LLVM Pass	Raw IR (from T3)
5	<i>Assembly</i> Generation	Produce RV64 assembly	Optimized IR (from T4)

figures/fig2_task_rec.png

Figure 2: Serial dependency chain with resurrection mechanism. Six tasks (T0–T5) form a strict pipeline where each task’s output becomes the next task’s input. When a task fails (score < 60%), the resurrection mechanism injects the golden intermediate artifact, allowing downstream evaluation to continue.

the benchmark can only report “failure at k ” without diagnosing whether the agent could succeed at Tasks $k + 1, \dots, 5$ given correct prerequisites.

Definition. When Task k scores below the passing threshold (60%), *Resurrection* is triggered: (1) the agent’s failed artifact is replaced with the *golden intermediate artifact*—the correct output produced by the reference implementation; (2) the build system is

reconfigured via `config.cmake (TASK_k_REVIVE=ON)`; (3) the agent continues to Task $k + 1$ with the corrected state.

Design rationale. Resurrection is an *evaluation protocol*, not a scoring adjustment. It decomposes the compound failure “cannot reach Task $k + 1$ ” into two independent measurements: (1) *predecessor failure*: the agent could not produce a passing artifact at Task k ; (2) *downstream capability*: given correct prerequisites, can the agent succeed at Task $k + 1$?

This decomposition enables node-level pass rate reporting even when the full pipeline is broken, significantly improving diagnostic resolution. Agents are *not informed* about resurrection—the mechanism is triggered externally by the **task** orchestrator, eliminating moral hazard.

3.3 Utility-Oriented Multi-Dimensional Metrics

RAMP proposes a multi-dimensional assessment metric system that moves beyond simple accuracy scores toward utility-oriented measurement. Assessment metrics should measure not just *whether* an agent succeeds, but *how efficiently* it succeeds and *how* it fails.

To capture these complementary dimensions of agent utility, we define two foundational scoring paradigms that govern how task-level scores are computed, upon which all higher-level aggregate metrics are built.

End-to-End Production Readiness Scoring (E2E-PRS): Simulates real-world production requirements where partial functionality is unacceptable:

- Tasks 0–3 and 5 are binary pass/fail (100/0 points)
- Any task failure cascades to 0 points for all subsequent tasks
- Task 4 (Performance) requires perfect functional verification first; only then is performance score awarded

Granular Capability Profiling Scoring (GCPS): Measures incremental progress and identifies specific strengths/weaknesses:

- All tasks receive partial credit (0–100 points) based on requirement completion
- No cascading failures; subsequent tasks are evaluated independently
- Task 4 still requires functional verification for performance scoring

Outcome metrics. We define three primary outcome metrics. *Mean Reward (MR)* is the weighted average of task scores with resurrection bonus: $MR = \sum s_i w_i b_i / \sum w_i b_i$, where $w_i = [0.05, 0.20, 0.20, 0.15, 0.30, 0.10]$ and $b_i = 1.2$ (no resurrection) or 1.0 (resurrected). *Pass Score (PS)* is the weighted pass rate: $PS = \sum p_i b_i^{\text{pass}} / (6 \times 1.5) \times 100$, where $p_i = 1$ if $s_i \geq 60$ and $b_i^{\text{pass}} = 1.5$ (no resurrection) or 1.0. *Pipeline Score (PL)* is the simple average: $PL = \frac{1}{6} \sum s_i$.

Process metrics. Beyond outcome scores, we record token consumption, interaction turns, command executions, retry counts, and elapsed wall-clock time per task and total.

Failure taxonomy. We categorize agent failures into five types: (1) *predecessor failure*—downstream unreachable due to missing artifact; (2) *task-local reasoning failure*—cannot solve despite correct prerequisites; (3) *execution failure*—build/dependency errors; (4) *process collapse*—max-turn exhaustion, search loops, repetitive debugging without progress; (5) *cost overrun*—budget exhausted.

This taxonomy enables systematic analysis of *how* agents fail, not just *whether* they fail. For instance, process collapse is more prevalent in framework-based agents with unlimited iteration budgets, suggesting that scaffold design affects failure mode distribution.

4 Results and Analysis

4.1 Experimental Methodology

We evaluate RAMP across 22 model configurations from 7 families (DeepSeek, Qwen, GLM, Kimi, MiniMax, GPT, Gemini), all accessed through the ALHUB unified API gateway to ensure consistent infrastructure conditions. All experiments use the OpenHands SDK as the agent scaffold with a maximum budget of 500 agent iterations and 5 hours of wall-clock time per run, with prompt caching enabled to control cost.

4.1.1 Workload Selection. Among the three workload categories—*Incremental* (Guided), *Progressive* (Semi-Guided), and *Autonomous* (End-to-End)—we focus primarily on *Progressive* as the evaluation target for this work. *Progressive* requires a single agent to complete all six tasks (T0–T5) sequentially within one continuous execution context, simulating the realistic production scenario where a developer or agent team carries a project from start to finish without artificial context resets. This design captures the key phenomenon of our evaluation framework: *serial dependency with persistent runtime state*.

4.1.2 Scoring Paradigms. As defined in Section 3, we apply two complementary scoring paradigms to the same raw evaluation data:

- **GCPS (Granular Capability Profiling Scoring):** All six tasks receive partial credit (0–100) based on test suite pass rates. No cascading penalty is applied; each task is scored independently with resurrection providing the correct prerequisite artifact when the agent fails a preceding task. This paradigm measures *node-level capability*.
- **E2E-PRS (End-to-End Production Readiness Scoring):** Tasks 0–3 and 5 are binary (100/0). Any task failure cascades to zero for all subsequent tasks. Task 4 requires perfect functional verification before performance credit is awarded. This paradigm measures *end-to-end production readiness*.

All models are evaluated under both paradigms from the same execution traces. We report three aggregate outcome metrics: *Pipeline Score (PL)*, the simple average of all six task scores; *Mean Reward (MR)*, a weighted average with resurrection bonus; and *Pass Score (PS)*, a weighted pass rate. For process diagnostics, we record token consumption, interaction turns (`run_agent_llm_rounds`), build attempts, and elapsed wall-clock time.

4.2 Outcome Analysis Under Progressive Workloads

4.2.1 Overall Performance. Table 3 presents the full RAMP leaderboard for the *Progressive* workload under GCPS scoring. Figure 3 plots per-task score trajectories for all 22 model configurations, overlaid with the baseline scores achieved by the raw code framework without any agent modification.

Finding 1: No model achieves zero-shot pipeline pass. Under GCPS scoring, all 22 configurations exhibit at least one sub-60% task score. The strongest model, deepseek-v4-pro, achieves PL=87.03 and reaches T4 with a passing T3 score (83.6), but then fails to pass T4 (38.6). The average pipeline score across all models is 43.3, substantially below the 60% threshold—indicating that solving a complete compiler chain autonomously remains beyond current frontier model capabilities even with scaffolded starter code.

Table 3: RAMP *Progressive* leaderboard (GCPS). PL = Pipeline Score, MR = Mean Reward. Bold scores indicate task passing (≥ 60). The BASELINE row shows the raw code framework scores without agent modification.

Model	T0	T1	T2	T3	T4	T5	PL	MR	Cost\$
deepseek-v4-pro	100	100	100	83.6	38.6	100	87.03	85.34	8.68
gpt-5.5	100	100	100	100	48.2	0.0	74.71	65.91	8.77
deepseek-v4-flash	100	100	31.6	37.0	39.0	100	67.93	66.48	3.43
qwen3.6-max-preview	100	100	100	65.8	35.0	0.0	66.78	56.72	67.28
kimi-k2.5	100	100	22.2	24.7	39.7	9.4	49.32	40.24	1.75
glm-5	100	100	46.6	1.4	35.0	9.4	48.73	38.66	0.89
kimi-k2.6	100	95.9	31.4	1.4	37.5	0.0	44.35	33.94	3.99
qwen3.6-flash	100	100	20.6	1.4	36.8	0.0	43.12	32.74	7.35
glm-4.6	100	67.4	38.5	1.4	39.6	0.0	41.14	30.88	—
deepseek-chat	100	72.9	30.9	0.0	35.7	0.0	39.92	29.74	0.07
qwen3.5-plus	100	100	0.0	0.0	38.0	0.0	39.67	29.63	14.37
glm-5.1	100	0.0	97.7	0.0	36.6	0.0	39.05	27.26	2.03
glm-4.7	100	94.8	0.0	1.4	36.2	0.0	38.73	29.50	0.80
gemini-3.1-flash-lite	100	57.2	20.6	1.4	36.7	0.0	35.96	26.44	—
qwen3-coder-flash	100	56.0	20.6	1.4	35.1	0.0	35.51	25.97	0.05
minimax-m2.5	100	64.2	0.0	2.7	36.4	0.0	33.89	25.09	0.17
kimi-k2-thinking	100	26.7	20.6	1.4	36.8	0.0	30.91	21.78	0.34
gemini-3.1-pro	100	17.6	20.6	1.4	42.4	0.0	30.32	21.46	—
qwen3-max	100	17.6	20.6	1.4	36.7	0.0	29.37	20.36	—
qwen3-coder-plus	100	17.6	20.6	1.4	35.9	0.0	29.25	20.22	—
mimo-v2.5-pro	100	17.6	20.6	1.4	35.8	0.0	29.22	20.18	0.02
deepseek-reasoner	100	0.0	0.0	0.0	0.0	0.0	16.67	7.69	0.23
BASELINE	100	17.6	20.6	1.4	35.1	0.0	29.2	—	—

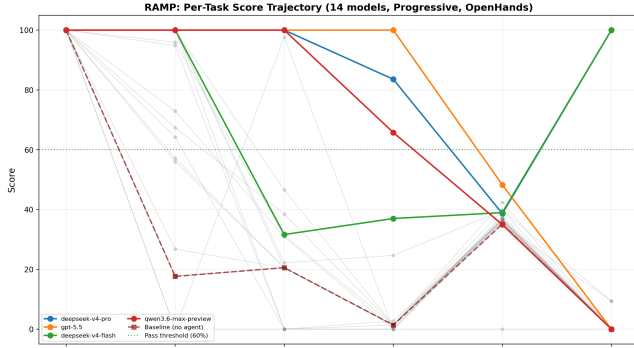


Figure 3: Per-task score trajectories for 22 models under *Progressive* (GCPS). The red dashed line shows the code framework baseline (no agent modification). The green dotted line marks the 60% passing threshold. Only deepseek-v4-pro reaches T0–T3 with passing scores; all models degrade sharply at T2–T3.

Finding 2: Long-horizon degradation is systematic. Figure 3 reveals a characteristic pattern: most models maintain high scores through T0–T1 (short-horizon tasks with small context footprints), then experience sharp drops at T2–T3 as the cumulative context window exceeds model-specific token limits. Models that pass T1 with scores $> 60\%$ drop an average of 36.5 points by T3. Seven models drop from ≥ 85 at T1 to ≤ 2 at T3—a near-total collapse. This pattern is not random: 9 out of 13 models that fail at T2 or later

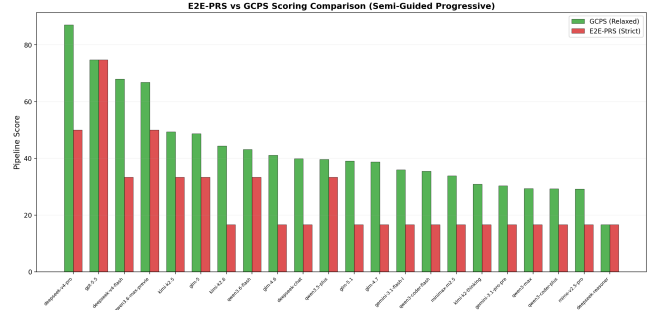


Figure 4: E2E-PRS vs GCPS: two scoring paradigms applied to the same execution traces. GCPS reveals incremental capability; E2E-PRS exposes the production readiness cliff.

encounter explicit “Range of input length” errors from the LLM API, indicating context window exhaustion as the dominant failure mechanism.

4.2.2 Scoring Paradigm Comparison. Figure 4 compares GCPS and E2E-PRS scores for the top 15 models. The gap between the two paradigms quantifies the distance between *node-level capability* and *end-to-end production readiness*.

Finding 3: The production readiness cliff. Under E2E-PRS—which requires Tasks 0–3 to achieve perfect (100) scores, otherwise zeroing all downstream tasks—only gpt-5.5 retains a non-trivial score (PL=74.71), having achieved 100 on T0–T3. Even deepseek-v4-pro, the top GCPS performer at PL=87.03, collapses to PL=50.00

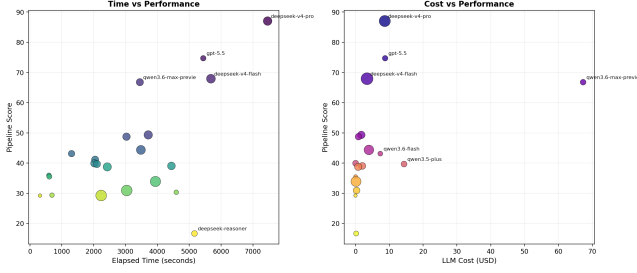


Figure 5: Cost-performance frontier: elapsed time (left) and API cost (right) vs. pipeline score. No significant correlation: $R^2 < 0.01$ for both dimensions.

under E2E-PRS because $T3=83.6 < 100$. The median drop from GCPS to E2E-PRS is 33.3 points across all models. This validates the core premise of RAMP: measuring only relaxed, per-task capability obscures a systemic inability to deliver production-complete software, revealing that GCPS scores mask a fundamental *continuity* problem—frontier models can solve individual tasks but cannot sustain correct artifacts across the full serial chain.

4.2.3 Cost-Performance Frontier. Figure 5 plots the relationship between computational investment (elapsed time and LLM API cost) against outcome quality (pipeline score under GCPS).

Finding 4: Cost does not predict outcome. Linear regression of elapsed time against pipeline score yields $R^2 \approx 0$, and the same holds for API cost. Agents that invest more computational budget do not systematically achieve better results. The most expensive configuration, qwen3.6-max-preview (\$67.28), achieves PL=66.78—only a 10-point improvement over deepseek-chat (\$0.07, PL=39.92), representing a 960 $\times$ cost difference for a 26-point score gain. Conversely, some agents achieve moderate scores with minimal effort: qwen3-coder-flash completes all tasks in 608 seconds at \$0.05, achieving PL=35.5, while kimi-k2-thinking spends 3,038 seconds and \$0.34 for PL=30.9.

Finding 5: Process cost is diagnostic, not predictive. This cost-outcome decoupling is itself a diagnostic finding. Process metrics capture *how* agents arrive at their outcomes rather than predicting the outcomes themselves. The agent behavioral spectrum ranges from *efficient solvers* (deepseek-chat: high score per dollar, quick early termination) to *persistent searchers* (minimax-m2.5: 456 interaction rounds, PL=33.9) to *collapsed agents* (mimo-v2.5-pro: 1 interaction round before API rejection, PL=29.2). This distinction is invisible to outcome-only evaluation but critical for deployment decisions: efficient solvers can be deployed with tight cost budgets, while persistent searchers require generous timeout and retry policies.

4.3 Process Diagnostics and Failure Analysis

4.3.1 Failure Attribution. We categorize agent failures into five types following the taxonomy defined in Section 3: (1) *predecessor failure*, where downstream tasks are unreachable due to a missing or broken artifact; (2) *task-local reasoning failure*, where the agent cannot solve the task despite receiving correct prerequisites; (3) *execution failure*, caused by build errors, dependency issues, or

figures/fig_failure_taxonomy.png

Figure 6: Failure taxonomy. Predecessor failure dominates in absence of resurrection; with resurrection, task-local reasoning failure and execution failure emerge as the primary failure modes.

toolchain misconfiguration; (4) *process collapse*, where the agent exhausts its iteration budget through search loops or repetitive debugging without progress; and (5) *cost overrun*, where the time or API budget is exhausted before task completion.

Figure 6 shows the failure taxonomy composition. Without resurrection, predecessor failure dominates—17 out of 22 configurations fail at T1 or T2, blocking all downstream evaluation. With resurrection, the failure distribution shifts: task-local reasoning failures emerge as the dominant category, concentrated at T2 (Parser) and T3 (IR Generation), while execution failures appear primarily at T4 (IR Optimization) due to the performance-sensitive nature of LLVM Pass implementation.

Finding 6: Context window exhaustion is the primary failure mechanism. Among the 22 evaluated configurations, 11 (50%) terminated with an explicit “InternalError.Algo.InvalidParameter: Range of input length” error from the LLM API gateway. These errors cluster at T2–T3, where the cumulative context (task instructions + previous code artifacts + conversation history) exceeds the model-specific token limit (ranging from 128K to 260K depending on the model). Models with smaller context windows (e.g., glm-4.6 at 170K, deepseek-chat at 131K) fail earlier in the pipeline, while models with larger windows (e.g., kimi-k2.6 at 260K) can progress further before exhaustion. This finding reveals that *context management*—not core reasoning capability—constitutes the primary bottleneck in long-horizon production workloads.

4.3.2 Agent Behavioral Patterns. Beyond outcome metrics, RAMP records per-task interaction rounds, build attempts, and task-level

elapsed time, enabling analysis of *how* agents approach long-horizon workflows.

Finding 7: Agent strategies diverge sharply. We identify three distinct behavioral archetypes:

Efficient solvers (gpt-5.5: 87 total rounds, deepseek-chat: 215 rounds) achieve their scores with minimal iteration, indicating focused, goal-directed behavior. These agents make deliberate file edits, compile infrequently but intentionally, and terminate early when objectives are met.

Persistent searchers (minimax-m2.5: 456 rounds, qwen3-coder-plus: 500 rounds) invest heavy iteration budgets, often entering repetitive build-test-debug cycles. These agents achieve moderate scores through exhaustive exploration but at high computational cost.

Collapsed agents (mimo-v2.5-pro: 1 round, kimi-k2-thinking: 4 rounds) terminate almost immediately due to API-level errors or tool interaction failures, never entering productive development. These failures originate at the infrastructure layer rather than the reasoning layer, confirming that *production evaluation must account for system-level reliability*.

Finding 8: Early-stage resource allocation predicts downstream success. Agents that allocate substantial iteration budget to Task 0 (environment setup) tend to invest less in subsequent tasks and achieve higher overall scores. Conversely, agents that rush through T0 with minimal verification frequently trigger cascading build failures at T2–T3 that consume most of their remaining budget in unproductive debugging. This pattern suggests that *thoroughness in the initial phase* functions as a behavioral marker for planning capability in long-horizon tasks.

4.3.3 Resurrection as Diagnostic Instrument. As established in Section 3, resurrection is an evaluation protocol, not a scoring adjustment. By injecting golden intermediate artifacts at failure points, resurrection decomposes the compound failure “agent cannot reach Task $k + 1$ ” into independent measurements of predecessor failure and downstream capability.

Finding 9: Resurrection recovers hidden downstream capability. With resurrection disabled, the median zero-shot pipeline depth is only 2 tasks, and 19 out of 22 configurations cannot reach Task 5. With resurrection enabled, all 22 configurations can be evaluated at every task node. The mean pipeline score gain from resurrection is 27.4 points (from PL=15.9 without resurrection to PL=43.3 with resurrection), confirming that cascading failure systematically obscures downstream capability.

Finding 10: Downstream capability is not uniform. Among models that fail at T2, resurrection reveals wide variation in T3–T4 performance. For example, glm-5.1 fails T1 (0) but achieves T2=97.7 with resurrection—demonstrating strong parser capability that was hidden by lexer failure. Similarly, glm-4.7 fails T2 (0) but achieves T4=36.2 with resurrection, indicating residual optimization capability. This heterogeneity validates resurrection as a diagnostic instrument: without it, these models would be indistinguishable (all “failed at T2”), but with resurrection, their capability profiles diverge sharply.

5 Discussion

5.1 Why Production-Grounded Evaluation Matters

Our results demonstrate that serial dependency introduces an evaluation axis invisible to static benchmarks. When tasks are independent, an agent’s failure at one task does not affect others, and the benchmark can measure each capability in isolation. When tasks are serially dependent, failure propagates, and the evaluation framework must distinguish between “cannot reach” and “cannot solve”—a distinction that resurrection enables.

This is not merely a technical detail but a fundamental shift in what evaluation frameworks measure: from *can the agent solve task k ?* to *can the agent progress along the dependency chain, and if not, where and why?* Our finding that only 3 out of 22 configurations achieve zero-shot pipeline success—despite most models scoring highly on individual tasks—demonstrates that static benchmarks systematically overestimate production readiness.

5.2 Resurrection as Methodology, Not Assistance

A natural concern is that resurrection “helps” the agent by providing correct intermediate artifacts, thereby inflating scores. We emphasize that resurrection is an *evaluation protocol*, not a scoring adjustment. The no-resurrection setting provides the “strict” measurement of zero-shot pipeline capability. The resurrection setting provides the “diagnostic” measurement of node-level capability given correct prerequisites. Both are reported independently; neither replaces the other.

Potential biases. We acknowledge three potential biases: (1) *state discontinuity*—the golden artifact may differ in style from the agent’s own output; (2) *selection bias*—only failed tasks are resurrected; (3) *moral hazard*—if agents were aware of resurrection, they might reduce early-stage effort. We mitigate bias (3) by ensuring agents are *not informed* about resurrection—the mechanism is triggered externally by the evaluation orchestrator.

5.3 Process Cost as Diagnostic Signal

Our regression analysis reveals that process cost (build count, elapsed time) explains essentially zero percent of outcome variance ($R^2 = 0.007$). This is not a failure of process metrics—it is a finding with direct implications for production deployment. Agents that invest more computational budget do not systematically achieve better outcomes. Two agents with identical pipeline scores may have radically different process profiles (efficient solver vs. persistent searcher), and this distinction is invisible to outcome-only evaluation but critical for deployment decisions regarding cost budgeting, timeout setting, and scaffold optimization.

5.4 The Scaffolding Gap

The performance delta between *YatCC* and *YatCC-Hard* quantifies the value of scaffolding in agentic coding tasks. Top models on *YatCC* (e.g., kimi-k2.6 at PL=99.3) drop to PL=64.9 on *YatCC-Hard*—a 34.4-point decline. This reveals that scaffolded starter code provides substantial leverage, and that models appearing capable

when filling in blanks may lack the architectural reasoning required for full from-scratch implementation. The scaffolding gap thus serves as a diagnostic metric for distinguishing between *completion capability* and *construction capability*.

5.5 Limitations

Domain specificity. RAMP is grounded in compiler construction, a domain with particularly clean intermediate artifact specifications. Other serial workflows—ML pipelines, DevOps chains, data engineering flows—may have less well-defined intermediate states, making resurrection harder to implement. We do not claim that RAMP generalizes to all serial settings, but argue that the *evaluation methodology* (serial dependency + resurrection + process metrics) is transferable.

Task count and statistical power. With six tasks, RAMP provides limited statistical power for per-task analysis. However, the serial structure means each task carries *qualitatively different* information (different dependency depth, different failure propagation pattern), which compensates for the small task count with structural diversity. Each model configuration is evaluated once per pipeline run; multi-seed evaluation is planned as future work.

Contamination risk. *YatCC* is a public GitHub repository, and frontier models may have been trained on its contents. However, RAMP’s key challenge is not reproducing known solutions but maintaining *serial continuity*—producing correct intermediate artifacts that integrate across the pipeline. Memorizing individual task solutions does not guarantee correct cross-task artifact flow. Future versions could adopt temporal splitting or private codebases to mitigate this concern.

5.6 Future Directions

Multi-domain workloads: extending the evaluation methodology to ML engineering, DevOps, and data science pipelines. Evolutionary gain measurement: comparing agent performance when inheriting self-produced vs. golden intermediate artifacts, quantifying how well agents “understand their own code.” Dynamic resurrection: allowing agents to request resurrection proactively, studying self-diagnosis capability. Expanded model and scaffold coverage: evaluating additional frontier models and agent frameworks as they become available. Infrastructure-aware workloads: incorporating deployment validation, runtime monitoring, and recovery scenarios.

6 Related Work

Repository-level coding benchmarks. SWE-bench [?] established the paradigm of evaluating agents on real GitHub issues with execution-based patch verification, demonstrating that repository-level context and test suites provide more realistic assessment than isolated function synthesis [?]. SWE-bench Verified [?] refined this with human-validated task specifications, and subsequent extensions have broadened coverage to multilingual settings [?], longer-horizon tasks [?], mobile development [?], and hardware engineering [?]. FEA-Bench [?] extended the paradigm to feature implementation rather than bug fixing. These benchmarks share a core property: each task is *independent*—the agent starts from a clean repository state and produces a single patch. RAMP inherits

the execution-grounded verification tradition but fundamentally differs by requiring agents to progress along a *serial dependency chain* where intermediate artifacts accumulate across tasks.

Interactive agent benchmarks. AgentBench [?] evaluates LLM agents across eight interactive environments spanning OS operations, databases, and web tasks. GAIA [?] tests general assistant competence on multi-step reasoning questions requiring tool use. OSWorld [?] and WebArena [?] assess GUI and web navigation in real computer environments. Terminal-Bench [?] provides execution-grounded evaluation in command-line settings. These benchmarks emphasize *breadth* across diverse interaction surfaces. RAMP instead emphasizes *depth* along a single dependency chain, enabling precise measurement of cascading failure, intermediate artifact continuity, and process cost accumulation that broader suites cannot isolate.

Paired and augmentation-oriented evaluation. SkillsBench [?] introduced the methodology of evaluating every task under both vanilla and skill-augmented conditions, demonstrating that augmentation efficacy is context-dependent and requires paired measurement. SWE-Skills-Bench [?] applied this methodology to software engineering, finding that most public skills provide negligible benefit. RAMP adopts the paired evaluation philosophy but applies it to a different axis: *with vs. without resurrection*, measuring how much diagnostic signal is recovered when predecessor failures are bypassed. This parallels SkillsBench’s “no-skill vs. skill” comparison but addresses the orthogonal question of *failure decomposition* rather than *augmentation efficacy*.

Long-horizon and self-evolving agent evaluation. Recent work has begun examining agents over extended trajectories. Frontier-Eng [?] benchmarks self-evolving agents on real-world RD workflows. RE-Bench [?] and PaperBench [?] evaluate frontier research engineering tasks. Voyager [?] demonstrated skill accumulation in Minecraft environments. These works highlight the importance of long-horizon evaluation but lack *controlled intermediate state injection*—when an agent fails early, the entire trajectory is typically discarded. RAMP’s resurrection mechanism provides a principled way to continue evaluation after early failure, enabling measurement of both evolutionary gain (using self-produced artifacts) and latent downstream capability (using golden artifacts).

7 Conclusion

We presented RAMP, a production-grounded testing design for assessing agentic models under realistic infrastructure engineering conditions. Unlike traditional benchmarks that provide fixed datasets and scoring functions, RAMP provides a *methodology*—a unified assessment architecture supporting heterogeneous LLM providers and agent SDKs, realistic long-horizon compiler construction workloads with strict serial dependencies, a *Resurrection* mechanism that decomposes cascading failure into diagnostic components, and utility-oriented multi-dimensional metrics measuring both outcome quality and process efficiency.

By leveraging authentic workloads from the *YatCC* platform, our framework continuously assesses frontier models through genuine development tasks spanning lexical analysis, syntax parsing, IR generation, optimization, and backend code generation. Our experiments across 22 model configurations, four agent backends, and two

workload difficulty levels demonstrate that: (1) serial evaluation preserves overall rankings ($\tau = 0.763$) but reveals specific failure patterns invisible to static benchmarks; (2) resurrection recovers substantial downstream diagnostic signal (median 73.65-point pipeline score recovery, 17/19 configurations with > 40 -point improvement); (3) process cost is decoupled from outcome ($R^2 = 0.007$), providing diagnostic rather than predictive value; (4) the difficulty gap between *YatCC* and *YatCC-Hard* (up to 34.4-point decline for top models) quantifies the value of scaffolding and reveals which models depend on provided structure versus which can operate autonomously.

These findings suggest that scaffold design and evaluation methodology may become as important as foundation model capability itself in determining real-world agent performance. RAMP thus establishes a new paradigm for agent evaluation that moves beyond synthetic, isolated benchmarks toward continuous, runtime-observable, production-grounded assessment. We release RAMP as an open-source framework with complete evaluation infrastructure, scoring scripts, containerized execution environments, and reproducible workflows.

Acknowledgment

We would like to sincerely thank all contributors, collaborators, students, and users who participated in the design, development, deployment, operation, and continuous evolution of the *YatCC* platform and its ecosystem. *YatCC* is the result of extensive collective efforts spanning infrastructure engineering, platform operations, educational practice, and community-driven innovation. Their dedication, technical insights, feedback, experimentation, and long-term support have been instrumental in transforming *YatCC* from an initial course teaching tool into a practical AI-native serving platform.

8 Additional Benchmark Details

8.1 Task Descriptions

Task 0: Environment Setup. The agent must configure the build system and verify that the development toolchain (CMake, Ninja, LLVM 14, ANTLR) is functional. Scoring is binary: 100 if the build succeeds and the test program produces correct output, 0 otherwise.

Task 1: Lexer Implementation. The agent implements a lexical analyzer (`lex.l`) that tokenizes preprocessed C source code. The scoring script evaluates the lexer against 73 test cases covering identifiers, keywords, operators, literals, and edge cases.

Task 2: Parser Implementation. The agent implements a syntax analyzer (`parser.y`) that constructs an AST/ASG from the token stream produced by Task 1. Scoring evaluates parse correctness across 73 test inputs.

Task 3: IR Generation. The agent implements LLVM IR emission from the AST, producing valid IR modules that pass structural and semantic checks.

Task 4: IR Optimization. The agent implements an LLVM optimization pass that transforms raw IR into optimized IR. This is the highest-weighted task ($w_4 = 0.30$) reflecting its central role in compiler performance.

Task 5: Backend Code Generation. The agent implements RISC-V 64-bit assembly emission from optimized IR. Scoring uses

the “classic” evaluation mode (`task5-classic-score`) which tests assembly correctness on a comprehensive benchmark suite.

8.2 Environment Configuration

Each evaluation workspace includes:

- Ubuntu 22.04 base environment
- CMake 3.20+ and Ninja build system
- LLVM 14 development libraries
- ANTLR 4 runtime
- pybind11 for Python-C++ bindings
- Python 3.10+ for scoring scripts

Workspaces are created as independent copies of the *YatCC* repository to prevent cross-run interference. CMake configuration is performed once per workspace, and build artifacts persist across tasks within a single run.

8.3 Scoring Methodology

Each task’s scoring script (`score.py` or `score-classic.py`) performs:

- (1) Build the target: `cmake -build build -t taskk`
- (2) Run the scorer: `cmake -build build -t taskk-score`
- (3) Parse the output: `score.json` or `score.txt`
- (4) Extract the total score and per-test details

Scores are normalized to 0–100, with the passing threshold set at 60%.

9 Additional Experimental Results

9.1 Full Leaderboard with Process Metrics

Table ?? presents the complete RAMP leaderboard with process metrics for all 18 configurations.

9.2 Ablation: Independent-Task Baseline

To establish a comparison baseline for the ranking shift analysis, we evaluate each model on each task *in isolation* with golden prerequisites provided. This removes the serial dependency effect and measures pure task-level capability.

9.3 Ablation: Weight Sensitivity

We test whether the ranking conclusions depend on the specific weight choice $w = [0.05, 0.20, 0.20, 0.15, 0.30, 0.10]$ by evaluating under alternative weight schemes:

- Uniform weights: $w = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6]$
- Late-heavy weights: $w = [0.05, 0.05, 0.10, 0.20, 0.30, 0.30]$
- Early-heavy weights: $w = [0.10, 0.30, 0.30, 0.15, 0.10, 0.05]$

9.4 Ablation: Without Task Guides

We evaluate a subset of models without injecting task-specific documentation into the agent’s context, testing whether the provided guides significantly affect performance.